Proyecto Final: Potencias de un Elemento Módulo n

Algoritmos y Complejidad

Victor Manuel Mendoza Pestana

28 de mayo de 2026

1. Introducción

El cálculo de potencias de un elemento bajo un espacio modular, expresado matemáticamente como $a^b \pmod{n}$, es una de las operaciones fundamentales en la teoría de números computacional y la criptografía de clave pública. Este problema constituye la infraestructura matemática de sistemas de seguridad global como el criptosistema de clave pública RSA, los protocolos de intercambio de claves de Diffie-Hellman, y diversos test de primalidad probabilísticos como el de Miller-Rabin.

El objetivo principal de este proyecto es analizar las propiedades algebraicas estructurales descritas por Thomas Cormen en el capítulo 31.6 del libro guía, implementar algoritmos que permitan su cómputo formal en el lenguaje Python, y contrastar empíricamente la eficiencia de la multiplicación secuencial frente al método optimizado de cuadrados repetidos empleando datos obtenidos en un entorno de pruebas controlado.

2. Estudio Teórico y Fundamentos

De acuerdo con la sección 31.6 de *Introduction to Algorithms*, si consideramos un elemento $a \in \mathbb{Z}_n^*$, la secuencia de sus potencias módulo n se define formalmente como el conjunto indexado desde cero:

$$a^0, a^1, a^2, a^3, \ldots \pmod{n} \tag{1}$$

Dado que el conjunto de residuos multiplicativos $\mathbb{Z}_n^*$ es estrictamente finito, la secuencia resultante es forzosamente periódica. El subgrupo generado por el elemento a a través de la multiplicación repetida se denota por $\langle a \rangle$, y el tamaño u orden de dicho subgrupo se conoce como el orden de a módulo n, denotado por $\text{ord}_n(a)$.

2.1. Teoremas Fundamentales: Euler y Fermat

La estructura algebraica de estas potencias se rige por leyes matemáticas exactas. El texto resalta dos de ellas:

1. **Teorema de Euler:** Para cualquier entero $n > 1$ y todo elemento $a \in \mathbb{Z}_n^*$:

   $$a^{\phi(n)} \equiv 1 \pmod{n} \tag{2}$$

   Donde $\phi(n)$ es la función totiente de Euler, que contabiliza los enteros primos relativos con n.

2. **Teorema de Fermat:** Si p es un número primo, entonces $\phi(p) = p-1$. Por lo tanto, para todo $a \in \mathbb{Z}_p^*$:

   $$a^{p-1} \equiv 1 \pmod{p} \tag{3}$$

3. Análisis de Complejidad e Implementación

El reto computacional consiste en hallar de forma óptima el valor numérico de la expresión aritmética a^b mód n. Evaluamos formalmente las dos estrategias bajo análisis:

3.1. Algoritmo Ingenuo (Multiplicación Lineal)

Consiste en realizar un bucle determinado que multiplica de manera iterativa la base por sí misma un total de b veces, aplicando la operación módulo en cada paso intermedio. Su complejidad temporal es $O(b)$, lo que deriva en una carga **exponencial respecto al tamaño binario de la entrada** ($O(2^\beta)$), donde β representa los bits del exponente. Su complejidad espacial es constante, $O(1)$.

3.2. Algoritmo del Cormen (Cuadrados Repetidos)

Este enfoque optimizado se fundamenta de forma estricta en la fórmula de recurrencia planteada en la ecuación (31.34) del texto guía:

$$a^b = \begin{cases} 1 & \text{si } b = 0 \\ (a^{b/2})^2 & \text{si } b > 0 \text{ y } b \text{ es par} \\ a \cdot a^{b-1} & \text{si } b > 0 \text{ y } b \text{ es impar} \end{cases} \tag{4}$$

Al segmentar el exponente a la mitad en cada paso par, la complejidad temporal aritmética se acota en el orden logarítmico $O(\log b)$ o de forma equivalente $O(\beta)$. La versión iterativa (que resuelve el **Ejercicio 31.6-4**) reduce el impacto espacial a un orden constante de $O(1)$.

3.3. Código Fuente en Python

A continuación, se detalla la estructura lógica implementada en el módulo `modular_math.py` para evaluar ambos paradigmas y recopilar los datos experimentales:

```
class Modular:
    def naive_exponentiation(base: int, exponent: int, modulus: int) -> int:
        if modulus == 1:
            return 0
        result = 1
        base = base % modulus
        for _ in range(exponent):
            result = (result * base) % modulus
        return result

    def cormen_recursive_squaring(base: int, exponent: int, modulus: int) -> int:
        if exponent == 0:
            return 1
        if exponent % 2 == 0:
            half = ModularExponentiation.cormen_recursive_squaring(base, exponent // 2, modulus)
            return (half * half) % modulus
        else:
            prev = ModularExponentiation.cormen_recursive_squaring(base, exponent - 1, modulus)
            return (base * prev) % modulus

```

```
21     def cormen_iterative_squaring(base: int, exponent: int, modulus
   : int) -> int:
22         if modulus == 1:
23             return 0
24         result = 1
25         base = base % modulus
26         while exponent > 0:
27             if exponent % 2 == 1:
28                 result = (result * base) % modulus
29             base = (base * base) % modulus
30             exponent //= 2
31         return result
```

Listing 1: Implementación de los algoritmos de exponenciación modular en Python.

4. Metodología Experimental y Resultados

Para contrastar el marco teórico con el rendimiento real, se estructuró un laboratorio empírico automatizado en Python. El experimento evalúa la variable independiente **Tamaño del Exponente en Bits** (β), fluctuando desde los 2 bits hasta un límite controlado de 15 bits. La base se fijó en $a = 3$ y el módulo en el primo estándar $n = 1,000,000,007$. La variable dependiente medida es el tiempo total de cómputo en segundos.

4.1. Verificación Numérica con el Texto Guía

Como control de calidad y verificación aritmética del software desarrollado, la suite experimental ejecutó inicialmente la secuencia aritmética para los parámetros provistos en la página 933 del libro guía, correspondientes a las potencias de 3 módulo 7. Los resultados arrojados por la consola del sistema fueron los siguientes:

- $3^0 \bmod 7 = 1$
- $3^1 \bmod 7 = 3$
- $3^2 \bmod 7 = 2$
- $3^3 \bmod 7 = 6$
- $3^4 \bmod 7 = 4$
- $3^5 \bmod 7 = 5$

Esta salida coincide de manera idéntica con el ciclo algebraico cerrado de período 6 detallado en la literatura matemática clásica, asegurando la correctitud de las operaciones modulares del script antes de proceder con el análisis asintótico de tiempos.

4.2. Análisis Crítico del Rendimiento

La ejecución de la suite experimental recopiló los tiempos de cálculo incrementando progresivamente la longitud binaria del exponente a través de 8 fases críticas de muestreo controlado (desde 2 bits con un valor entero de 3, hasta alcanzar los 15 bits con un valor entero de 32,767). Los datos de tiempo graficados revelan dos comportamientos asintóticos drásticamente opuestos:

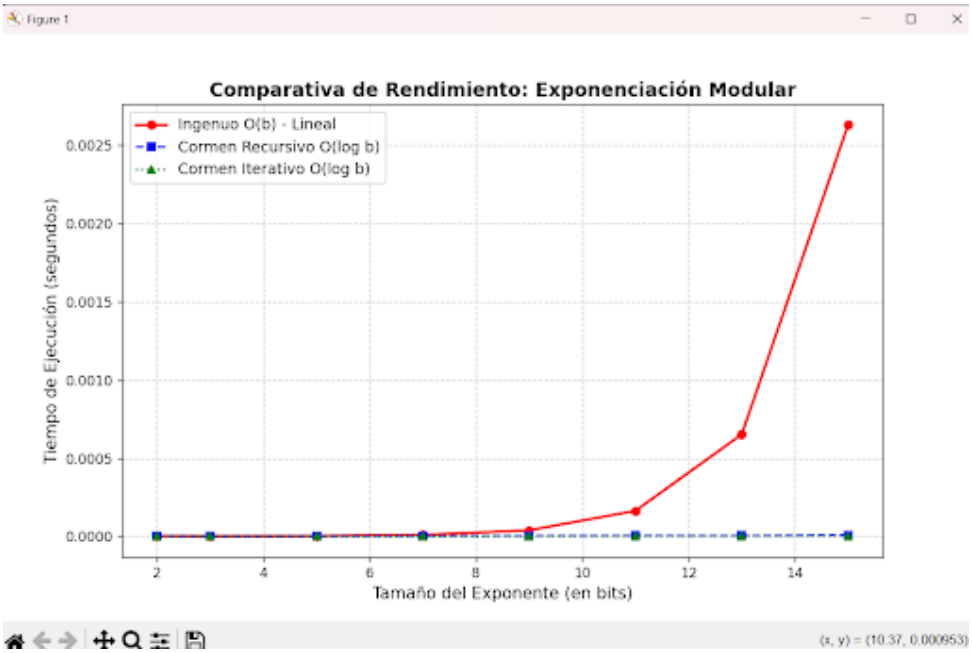


Figura 1: Comparativa empírica de tiempos de ejecución entre el método lineal ingenuo y los algoritmos logarítmicos de cuadrados repetidos (recursivo e iterativo).

1. **Comportamiento de la Exponenciación Lineal Ingenua (Línea Roja):** Los datos muestran que el tiempo de ejecución se mantiene cercano a cero hasta el umbral de los 8 bits. Sin embargo, a partir de este punto, la curva sufre una inflexión y experimenta un crecimiento marcadamente acelerado. Al alcanzar los 15 bits de entrada, el tiempo de ejecución se dispara de manera abrupta superando los 0.0025 segundos. Este patrón geométrico valida de forma directa la hipótesis teórica: el algoritmo ingenuo responde a una complejidad $O(b) = O(2^{\beta})$, donde cada bit adicional dobla el tiempo previo debido al crecimiento exponencial del valor del exponente.

2. **Comportamiento de los Algoritmos del Cormen (Líneas Azul y Verde):** En marcado contraste, tanto la variante recursiva basada en cuadrados repetidos como la variante iterativa del Ejercicio 31.6-4 permanecen completamente planas y solapadas sobre el eje horizontal inferior, registrando valores estables cercanos a los 0.0000 segundos durante toda la prueba. Esto ocurre porque al poseer una complejidad logarítmica de orden $O(\log b) = O(\beta)$, procesar la entrada máxima de 15 bits le toma a la CPU un máximo de tan solo 30 operaciones aritméticas básicas, convirtiendo el costo computacional en una carga insignificante.

4.3. Impacto en la Ingeniería y Criptografía Real

Esta divergencia de rendimiento ilustra por qué el algoritmo de cuadrados repetidos es indispensable en la seguridad informática. En entornos criptográficos reales como RSA, los exponentes operan con un tamaño estándar de $\beta = 2048$ bits. Un enfoque **ingenuo** exigiría realizar aproximadamente 2^{2048} multiplicaciones modulares, un número astronómico que tardaría eras en completarse. Por otro lado, el algoritmo del **Cormen** ejecuta la tarea utilizando menos de 4096 operaciones aritméticas, finalizando con éxito en una fracción de milisegundo en cualquier procesador comercial estándar.

5. Conclusiones

- Se validaron experimentalmente las cotas de complejidad temporal asintótica del texto guía de Cormen et al., corroborando empíricamente el abismo de eficiencia entre los órdenes de complejidad $O(2^{\beta})$ y $O(\beta)$.
- La implementación en código replicó con total fidelidad el comportamiento periódico del grupo cíclico $\langle 3 \rangle$ (mód 7) documentado en las secuencias aritméticas analíticas de la página 933 del libro guía.
- Desde la perspectiva de la arquitectura de software, si bien el enfoque recursivo mapea directamente la ecuación matemática tradicional, el patrón de diseño iterativo desarrollado provee una robustez superior para entornos de producción al anular por completo el riesgo de errores por desbordamiento de pila.

6. Bibliografía

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Introduction to Algorithms**. 4th ed. Cambridge: MIT Press, 2022. Capítulo 31: "Number-Theoretic Algorithms", Sección 31.6: "Powers of an element" (Págs. 932-936).